

# **UNIVERSITI PUTRA MALAYSIA**



## **CCS3402 – Database Application Development**

### **Lab 2**

#### **PREPARED BY :**

Nay Zin Min Lwin (231198)

**SESSION:** 2025/2026

Question: Registration form using Java Swing.

## Program

### App.java

```
package me.alex;

import javax.swing.SwingUtilities;

/**
 * Application entry point.
 */
public final class App {

    private App() {

    }

    public static void main(String[] args) {
        // Build and show the registration UI on the Swing event thread.
        SwingUtilities.invokeLater(() -> {
            PrimaryController form = new PrimaryController();
            form.setVisible(true);
        });
    }
}
```

### DatabaseConfig.java

```
package me.alex;

/**
 * Centralized Oracle database settings used by the registration form.
 */
public final class DatabaseConfig {

    private DatabaseConfig() {

    }

    /**
```

```

* JDBC connection string. Example: jdbc:oracle:thin:@localhost:1521/ORCLPDB1
*/
public static final String JDBC_URL = "jdbc:oracle:thin:@fsktmdbora.upm.edu.my:1521/fsktm";

/** Oracle username. */
public static final String USERNAME = "G231198";

/** Oracle password. */
public static final String PASSWORD = "231198";
}

```

## PrimaryController.java

```

package me.alex;

import javax.swing.BorderFactory;
import javax.swing.ImageIcon;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.WindowConstants;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.awt.event.ActionEvent;
import java.net.URL;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

/**
 * Main Swing registration form and submission handler.
 */

```

```

public class PrimaryController extends JFrame {

    // SQL statement used to insert one user record.
    private static final String INSERT_SQL = "INSERT INTO java_registered_users (name, email, phone,
address, password) VALUES (?, ?, ?, ?, ?)";

    private final JLabel headerIcon = new JLabel();
    private final JTextField nameField = new JTextField(25);
    private final JTextField emailField = new JTextField(25);
    private final JTextField phoneField = new JTextField(25);
    private final JTextField addressField = new JTextField(25);
    private final JPasswordField passwordField = new JPasswordField(25);
    private final JPasswordField confirmPasswordField = new JPasswordField(25);

    public PrimaryController() {
        super("Create a new account");
        buildUi();
        loadHeaderIcon();
    }

    // Loads the header icon from the resources folder.
    private void loadHeaderIcon() {
        URL iconUrl = getClass().getResource("/image/new_user_icon.png");
        if (iconUrl != null) {
            headerIcon.setIcon(new ImageIcon(iconUrl));
        }
    }

    // Builds the registration window layout and wires button events.
    private void buildUi() {
        setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        // Thin colored strip at the top for visual emphasis.
        JPanel top = new JPanel(new BorderLayout());
        top.setBackground(new Color(38, 70, 83));
        top.setBorder(BorderFactory.createEmptyBorder(8, 8, 8, 8));
        add(top, BorderLayout.NORTH);

        // Main form container uses GridBagLayout for aligned rows.
        JPanel formPanel = new JPanel(new GridBagLayout());
        formPanel.setBorder(BorderFactory.createEmptyBorder(18, 24, 18, 24));
        GridBagConstraints gbc = new GridBagConstraints();

```

```

gbc.insets = new Insets(8, 8, 8, 8);
gbc.anchor = GridBagConstraints.WEST;
gbc.fill = GridBagConstraints.HORIZONTAL;

// Header row: icon + title text.
JLabel title = new JLabel("Register");
title.setFont(new Font("Segoe UI", Font.BOLD, 24));
title.setHorizontalAlignment(SwingConstants.LEFT);

gbc.gridx = 0;
gbc.gridy = 0;
formPanel.add(headerIcon, gbc);

gbc.gridx = 1;
formPanel.add(title, gbc);

// Input rows for user registration details.
addRow(formPanel, gbc, 1, "Name", nameField);
addRow(formPanel, gbc, 2, "Email", emailField);
addRow(formPanel, gbc, 3, "Phone", phoneField);
addRow(formPanel, gbc, 4, "Address", addressField);
addRow(formPanel, gbc, 5, "Password", passwordField);
addRow(formPanel, gbc, 6, "Confirm password", confirmPasswordField);

// Action buttons for submit and close.
JPanel buttonPanel = new JPanel();
JButton registerButton = new JButton("Register");
registerButton.addActionListener(this::onRegister);

JButton cancelButton = new JButton("Cancel");
cancelButton.addActionListener(e -> onCancel());

buttonPanel.add(registerButton);
buttonPanel.add(cancelButton);

gbc.gridx = 0;
gbc.gridy = 7;
gbc.gridwidth = 2;
gbc.anchor = GridBagConstraints.CENTER;
formPanel.add(buttonPanel, gbc);

// Final window sizing and placement.

```

```

        add(formPanel, BorderLayout.CENTER);
        pack();
        setLocationRelativeTo(null);
        setResizable(false);
    }

    // Utility helper to add one label-input row to the form grid.
    private static void addRow(JPanel panel, GridBagConstraints gbc, int row, String labelText, JTextField
input) {
        gbc.gridwidth = 1;
        gbc.anchor = GridBagConstraints.EAST;
        gbc.gridx = 0;
        gbc.gridy = row;
        gbc.weightx = 0.0;
        panel.add(new JLabel(labelText), gbc);

        gbc.anchor = GridBagConstraints.WEST;
        gbc.gridx = 1;
        gbc.weightx = 1.0;
        panel.add(input, gbc);
    }

    // Validates form values and writes a new user record to Oracle.
    private void onRegister(ActionEvent ignored) {
        String name = trimOrEmpty(nameField.getText());
        String email = trimOrEmpty(emailField.getText());
        String phone = trimOrEmpty(phoneField.getText());
        String address = trimOrEmpty(addressField.getText());
        String password = passwordField.getPassword() != null ? new String(passwordField.getPassword()) :
"";
        String confirm = confirmPasswordField.getPassword() != null ? new
String(confirmPasswordField.getPassword())
        : "";

        // Basic required-field validation.
        if (name.isEmpty() || email.isEmpty() || phone.isEmpty() || address.isEmpty()
            || password.isEmpty() || confirm.isEmpty()) {
            showAlert(JOptionPane.WARNING_MESSAGE, "Missing data", "Please fill in every field.");
            return;
        }

        // Ensure password and confirmation match.
        if (!password.equals(confirm)) {

```

```

        showAlert(JOptionPane.WARNING_MESSAGE, "Password mismatch",
            "Password and Confirm password must match.");
        return;
    }

    // Stop early when placeholder DB credentials are still present.
    if (isPlaceholderConfig()) {
        showAlert(JOptionPane.WARNING_MESSAGE, "Database not configured",
            "Set JDBC_URL, USERNAME, and PASSWORD in DatabaseConfig.java to your Oracle
credentials.");
        return;
    }

    try (Connection conn = DriverManager.getConnection(
        DatabaseConfig.JDBC_URL, DatabaseConfig.USERNAME, DatabaseConfig.PASSWORD);
        PreparedStatement ps = conn.prepareStatement(INSERT_SQL)) {
        ps.setString(1, name);
        ps.setString(2, email);
        ps.setString(3, phone);
        ps.setString(4, address);
        ps.setString(5, password);
        int rows = ps.executeUpdate();
        if (rows == 1) {
            showAlert(JOptionPane.INFORMATION_MESSAGE, "Success", "User registered successfully.");
            clearForm();
        }
    } catch (SQLException e) {
        showAlert(JOptionPane.ERROR_MESSAGE, "Database error",
            "Could not save the user. Check your Oracle URL, credentials, and that the table exists.\n\n"
                + e.getMessage());
    }
}

// Closes the form window.
private void onCancel() {
    dispose();
}

// Normalizes nullable inputs to trimmed strings.
private static String trimOrEmpty(String s) {
    return s == null ? "" : s.trim();
}

```

```
// Detects unconfigured placeholder credentials.
private static boolean isPlaceholderConfig() {
    return DatabaseConfig.JDBC_URL.contains("YOUR_HOST")
        || DatabaseConfig.USERNAME.contains("YOUR_ORACLE_USERNAME")
        || DatabaseConfig.PASSWORD.contains("YOUR_ORACLE_PASSWORD");
}

// Shows a simple dialog message.
private void showAlert(int messageType, String title, String message) {
    JOptionPane.showMessageDialog(this, message, title, messageType);
}

// Clears all form fields after successful save.
private void clearForm() {
    nameField.setText("");
    emailField.setText("");
    phoneField.setText("");
    addressField.setText("");
    passwordField.setText("");
    confirmPasswordField.setText("");
}
}
```



## Output

Create a new account



# Register

Name

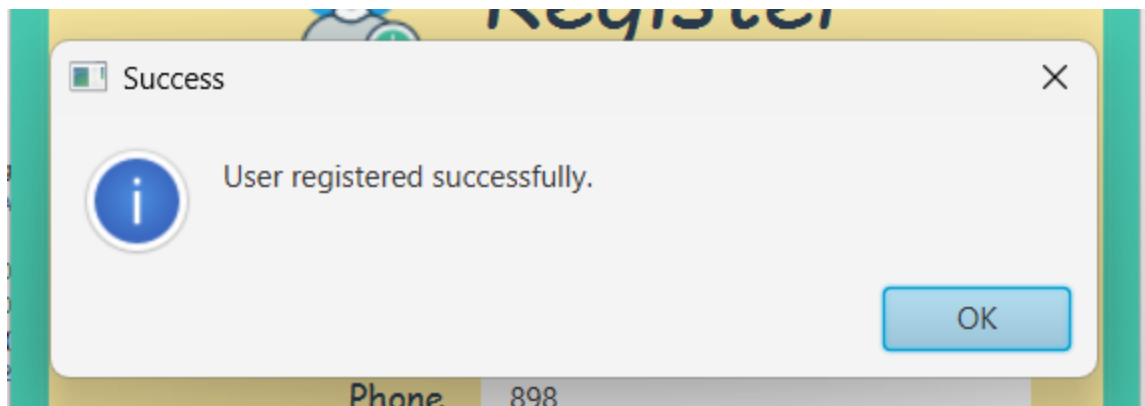
Email

Phone

Address

Password

Confirm password



7/11 Rows fetched: 1 in 0.000 seconds

|   | USERID | NAME | EMAIL        | PHONE | ADDRESS | PASSWORD |
|---|--------|------|--------------|-------|---------|----------|
| 1 | 1      | Alex | alex@upm.com | 898   | Serdang | 123      |

[Link](#)

[NayZinMinLwin\\_Github\\_Lab2\\_UserRegistration](#)